

COMMUNITY NETWORK — ORNITH BUILD SPEC

Purpose: This document is the single source of truth for the overnight implementation. Read it as a product + engineering specification. Do not invent features, change the core model, or turn this into a generic social-media app.

Product thesis: A hyperlocal network where people post what they **need, offer, plan, share, find, sell, borrow, or do**. The system connects relevant nearby people using **intent + proximity + AI matching + trust**.

Initial deployment: Graphic Era University and nearby area. Architecture must be reusable for other campuses/communities later.

1. NON-NEGOTIABLE PRODUCT PRINCIPLES

Principle	Requirement
Intent-first	Everything is a post/intent. Do not create unrelated mini-apps.
Hyperlocal	Every relevant post has a configurable geographic visibility radius.
Privacy	Never expose exact home/room location publicly. Show approximate distance/area.
AI is functional	AI classifies posts, extracts structured intent, matches people/posts, and flags risk indicators.
Human choice	AI recommends; the user decides. Never auto-send money or commit to an exchange.
Trust	XP measures participation; Trust measures reliability. They are separate.
No paid progression	No paid badges, XP, boosts, cosmetics, or premium trust.
Ephemeral where appropriate	Time-sensitive posts expire/close automatically.

2. CORE POST MODEL

Do **not** build separate systems for rides, marketplace, social, help, etc. They are post types on one common post model.

Post type	Example / behavior
Need Help	"Need Lenovo charger near campus."
Offer / Give	"Have spare Lenovo charger."
Borrow / Lend	"Need HDMI cable for 3 hours."
Group Buy	"Ordering 2 pizzas for ₹299; anyone wants one?"
Going / Ride	"Graphic Era → Saharanpur Chowk at 5 PM; 2 seats."
Plan / Join Me	"Going to a café at 7 PM; anyone want to join?"
Sell / Service	Jacket for ₹300; PPT design for ₹300; tutoring; printing.
Campus / Community	Campus tips, photos, places, alerts, fresher advice.
Opportunity	Internships, freelance work, volunteering, legitimate paid tasks.
Lost / Found	Physical item recovery; visual + location + time matching.

3. PROXIMITY + DISCOVERY

Each post may specify a visibility radius: **500 m, 1 km, 2 km, 5 km, 10 km, 25 km, Campus, or Custom**. Default should be sensible but user-controlled.

For route-based posts, support **route overlap**: a user travelling from A→B can match another user whose route substantially overlaps, even if they are not inside the same simple radius.

Discovery should support both a feed and a **Help Radar**: nearby active intents such as people needing help, giving items away, group buys, rides, plans, etc.

Public UI should show approximate distance/area, not exact private location. Recommend safe public meetup points for real-world exchanges.

4. AI INTENT ENGINE

User should be able to write naturally instead of filling a long form. AI converts free text into structured fields.

Example: "I'm leaving GEU around 5, going towards Saharanpur." → type=GOING, origin=GEU, destination=Saharanpur, time=5 PM, seats=unknown.

AI responsibilities: **classify post type; extract entities/time/location/price; generate tags; identify urgency; find semantically similar posts; rank nearby matches; detect route overlap; suggest expiry; identify potential safety/risk indicators.**

AI must not make irreversible decisions. Risk detection should say **"risk indicators detected"** with reasons, not declare a person a scammer solely from model output.

5. MONEY + GROUP EXPENSES

Posts involving money support: **Good Samaritan (free), equal split, fixed contribution, proportional contribution, custom amount.**

Plans/groups have an optional **shared expense manager**. Every expense stores payer, amount, description, beneficiaries, and whether it is shared or personal.

Beneficiaries may be selected individually. Example: Taxi ■500 → Shreya + Rahul + Atishay; Crochet ■200 → Shreya only.

At any point, each user can see: total personally paid, personal expenses, shared contributions, amount owed, amount receivable, and suggested settlement transfers. The app calculates; users choose whether/how to settle.

6. PLANS / HANGOUTS / SOCIAL

Use one **Plans** system instead of a separate dating product. A plan has location, time, capacity, budget, purpose, and optional vibe tags.

Purposes: Hangout, Food, Gaming, Study, Movie, Activity, Networking, Event, Dating, etc. No restrictive demographic filters such as "only BTech/hostelites"; users can participate based on the plan and safety rules.

Campus feed supports photos, tips, recommendations, alerts, fresher advice, places to visit, and general community posts. It must remain hyperlocal and useful rather than becoming a generic global social network.

7. MARKETPLACE / BORROW / SERVICES

Support **Buy/Sell, Free/Give, Borrow/Lend, Rent, Group Buy, and Services**. Examples: charger, A4 sheets, books, tripod, calculator, jacket, PPT design, tutoring, printing.

The core distinction is **Need ↔ Offer**: a user asking for something can be matched with someone offering it nearby.

8. OPPORTUNITIES + SAFETY

Opportunity posting is unlocked only for **Level 5 + verified users**. Categories: internship, freelance, part-time, volunteering, legitimate paid services/tasks.

Do not encourage academic cheating. Allow legitimate tutoring, debugging help, design, editing, printing, etc.

AI Safety Check should inspect signals such as unusual payment requests, suspicious external links, unrealistic guaranteed-money claims, account/post history, and community reports. Show evidence/reasons, not an unsupported verdict.

Provide report, block, moderation, and safe-meetup mechanisms. Real-world exchanges should encourage public meeting points.

9. CHAT

Built-in 1:1 and group chat. Users should not need to reveal phone numbers. Chat is attached to posts/plans/exchanges so context is preserved.

10. XP + TRUST + COSMETICS

XP: community participation. Examples: useful response, successful exchange, helping someone, returning borrowed item, useful community post.

Trust: reliability in real interactions. Signals include verified identity, completed exchanges, meetup completion, reports, cancellations, return history, and feedback attributes such as Reliable / Helpful / On time / Honest / Good communicator.

Levels: Newcomer → Local → Regular → Helper → Trusted → Connector → Community Pillar (names can change). Level gates Opportunity posting at Level 5.

Cosmetics: free profile cosmetics/badges earned through progression. Never sell XP, trust, badges, boosts, or progression.

11. POST LIFECYCLE

Posts must support states such as **ACTIVE** → **INTERESTED** → **MATCHED** → **IN PROGRESS** → **COMPLETED / EXPIRED / CANCELLED**.

Time-sensitive posts should expire automatically: rides after departure, group buys after order deadline, plans after event time, urgent requests after a chosen window. Marketplace posts remain active until sold/closed.

12. SAFETY / PRIVACY BASELINE

- No public exact room/home coordinates.
- Approximate location and configurable radius.
- Block/report users and posts.
- Moderation queue for flagged content.
- Safe public meetup suggestions.
- No automatic financial transactions.
- No automatic acceptance of strangers into a plan/ride.
- AI risk analysis is advisory and explainable.
- Private chat; phone number not required.

13. TECHNICAL CORE FOR OVERNIGHT BUILD

Recommended stack: Next.js + TypeScript + Tailwind; Supabase (Postgres, Auth, Storage, Realtime); Vercel deployment. Use an external AI API only where needed.

Core entities: User, Profile, Verification, Post, PostType, LocationArea, Route, Plan, GroupMember, Chat, Message, Expense, ExpenseParticipant, Transaction/Settlement, Report, TrustEvent, XPEvent, Cosmetic, Notification.

Post fields: author, type, title/body, media, structured AI fields, visibility radius, approximate location, optional route, time window, price/contribution mode, status, expiry, tags, created_at/updated_at.

Use geospatial querying where practical. Keep the data model generic enough that all post types use the same discovery/matching infrastructure.

14. MVP FOR THE HACKATHON

BUILD FIRST	Reason
Auth + profile + verification state	Foundation + trust
Create/view posts	Core product
AI post classification + structured extraction	Main AI differentiator
Configurable proximity	Core identity
Nearby feed + Help Radar	Discovery
AI matching	Main intelligence
Going/route post	Strong demo
Plans + joining	Social layer
Group expense calculator	Strong differentiator
1:1/group chat	Required interaction loop
Buy/sell/borrow/group-buy	Real-world utility
XP + Trust basics	Retention/safety
AI safety indicators	Required for marketplace/opportunities
Lost/found visual matching	Showcase of multimodal matching

15. POST-MVP / DO NOT LET THESE BLOCK THE DEMO

- Advanced route optimization.
- Complex payment gateway integration.
- Full automatic identity verification.
- Large-scale moderation infrastructure.
- Sophisticated recommendation model training.
- Native mobile apps.
- Complex dating algorithms.
- Full campus administration integrations.
- Real-money escrow.

16. DEMO STORY

Demo should tell one continuous story:

1. User opens the campus network. 2. Posts “Need Lenovo charger within 2 km.” 3. Nearby user receives/matches it. 4. User creates “Graphic Era → Saharanpur Chowk, 5 PM” and another user is matched through route overlap. 5. Create a hangout/food plan. 6. Group members add taxi/pizza/personal expenses; settlement is calculated. 7. A marketplace/borrow post is matched. 8. Show AI safety indicators on an opportunity. 9. Show XP/Trust update. 10. End on the principle: **Need something? Ask nearby. Have something? Share it. Going somewhere? Take someone. Doing something? Invite people.**

17. IMPLEMENTATION RULE FOR ORNITH

Do not hallucinate missing requirements. If a requirement is ambiguous, preserve the common post/intent architecture and choose the smallest implementation consistent with this document.

Do not replace the product with a generic social feed, marketplace, dating app, ride app, or chatbot. All features must connect to the central **Intent + Proximity + Matching + Trust** model.

Prioritize a working end-to-end system over incomplete advanced features. Seed realistic demo data where external real-world data is unavailable. Keep interfaces modular so the UI can be redesigned later without rewriting backend logic.

UI will be redesigned separately later. Build clean, functional, API/data-driven interfaces now; do not spend the overnight build budget on visual polish.

END OF SPEC — This document defines the intended scope. The next engineering step is to convert it into architecture, database schema, API contracts, and an ordered implementation plan.